

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 7 Computer Peripherals and I/O

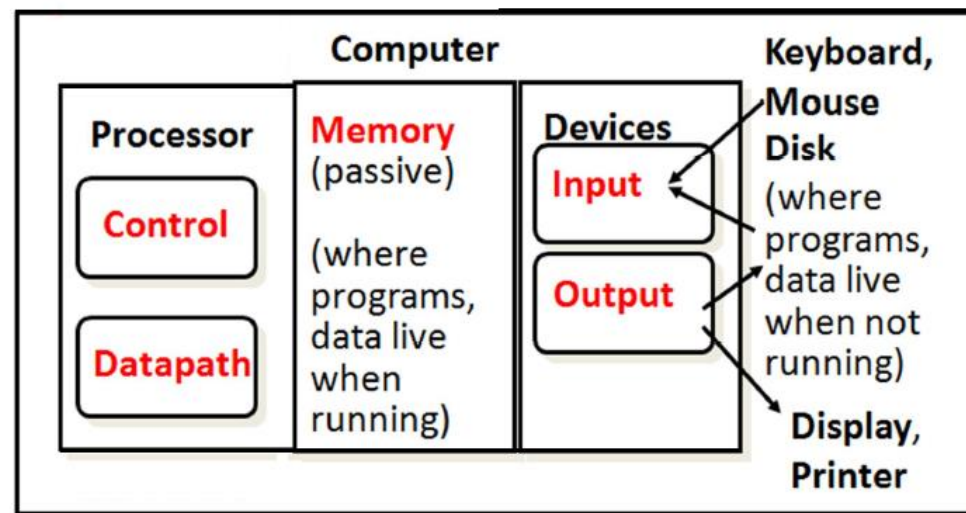
Disclaimer



- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Computer I/O

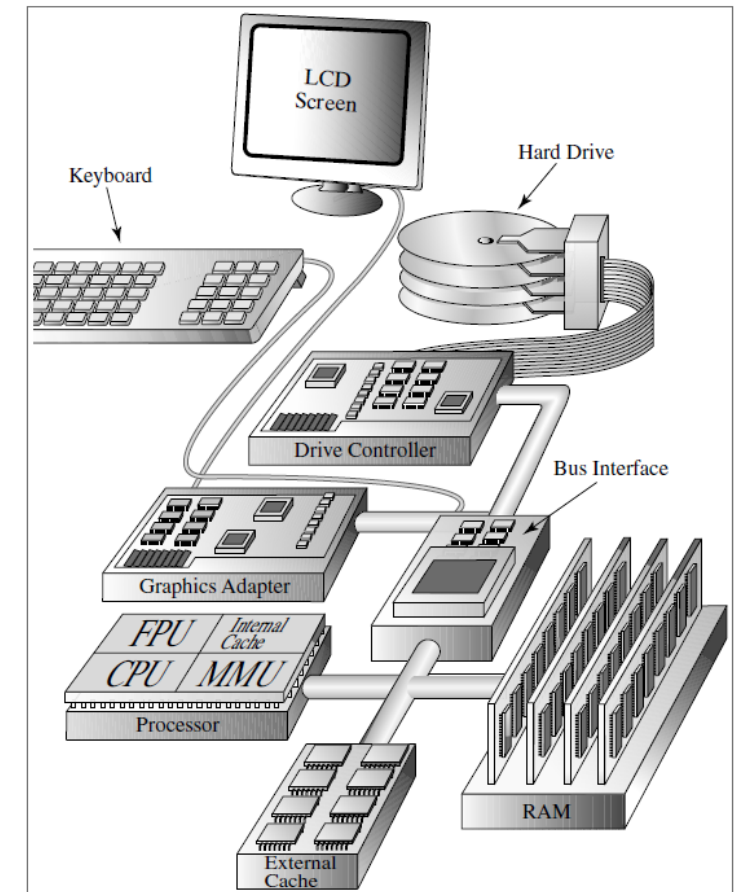
- I/O is one of the most complex areas of computer design
 - Wide variety of different I/O devices
 - The quest for fast handling of programs and data
- Two main purposes of I/O devices
 - To communicate with outside world
 - To store data



CD-ROM,
modems,
scanners, sound
cards with
speakers and
microphones, ...

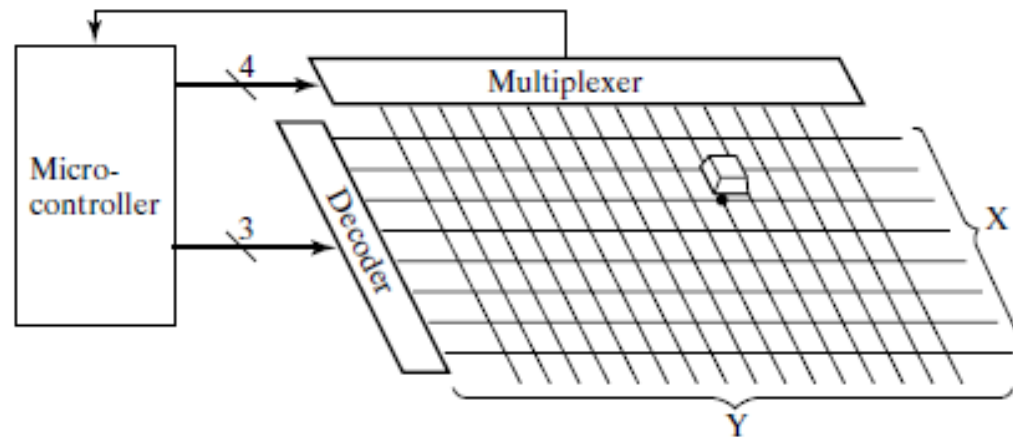
Peripherals

- I/O devices attached to the computer online are called peripherals.
 - Connected online = control by CPU.
- Example: Keyboard, hard drive, graphic display etc.



Example: Keyboard

- One of the slowest data rates.
- It is necessary to detect which of the keys have been depressed/ released.
 - A scan matrix lies beneath the keys.
 - Microcontroller is programmed to periodically scan all intersections in the matrix.
- Data transfer rate less than 10 bytes/s.



Keyboard Scan Matrix

I/O Interfaces

- The properties of peripherals and CPU & memory are different:

Peripherals	CPU & Memory
Electromechanical devices	Electronic devices
Usually slow data transfer rate	Usually faster than peripherals
Data unit = byte, block, ...	Data in word format

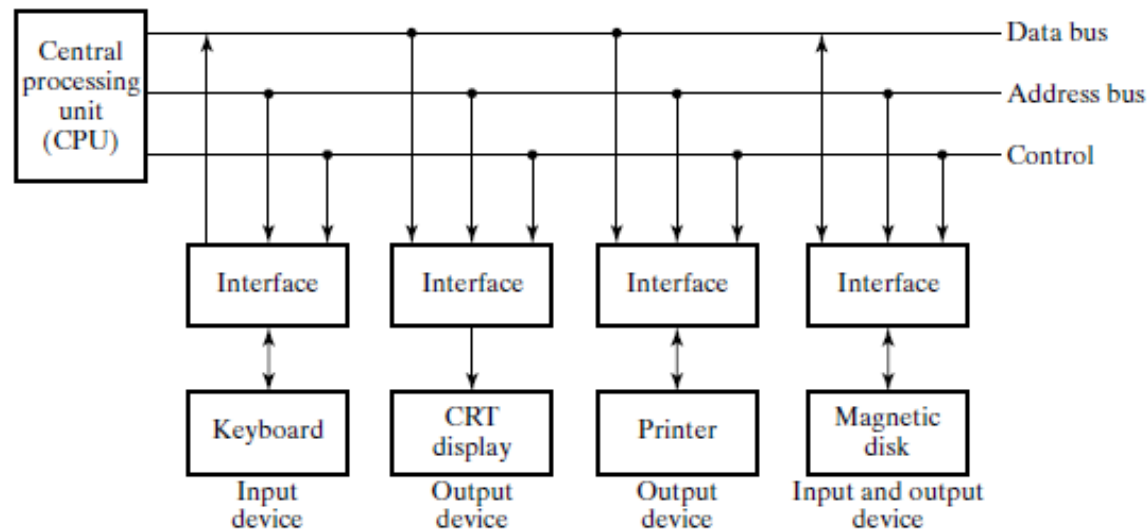
- The computer need special hardware to interface the peripherals with CPU: **Interface units**
 - Interface between the **bus** from the **CPU** and **peripheral devices**.
 - Supervise and synchronize all input and output transfer.
 - Each device has its own controller to supervise the operations of the particular mechanism of that peripherals
 - Example: the controller in a printer controls the motion of paper, timing of printing, the selection of the characters to be printed.

Buses

- Bus = a group of electrical signals / wires that carry these signals.
- Buses: internal and external
 - Internal bus is any bus within a processor system.
 - An external bus is mainly used as an I/O device interface.

I/O Bus and Interface unit

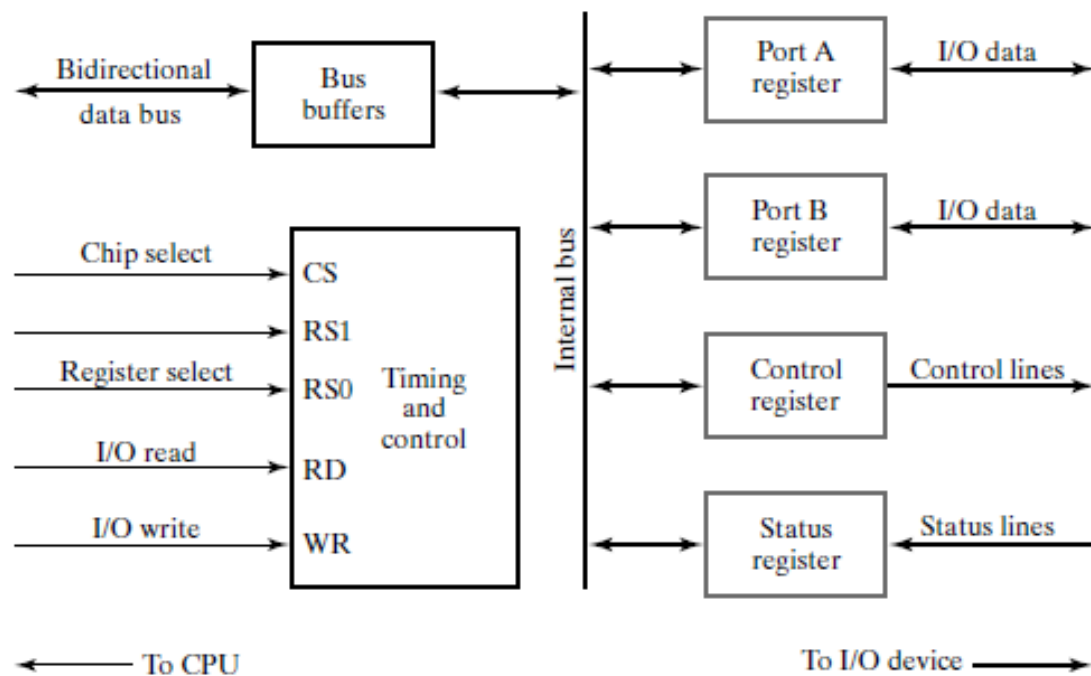
- The **three major components** of system bus:
 - **Data bus:** Carry data from source to destination (e.g., CPU to memory)
 - **Address bus:** Used to designate source or destination of the data.
 - **Control bus:** Carry functional information defining and controlling the current bus activity, e.g., clock, memory R/W, IO R/W, Bus request, etc.



Connection of I/O devices to CPU

Interface Unit

- Function of the interface:
 - Translate the signals between the CPU buses and the I/O devices.
 - Provide the needed hardware to satisfy the two sets of timing constraints



CS	RS1	RS0	Register selected
0	X	X	None: data bus in high-impedance state
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

Example of I/O Interface Unit

Interface Unit

- Typically, I/O controllers have three types of internal registers
 - Data
 - Command
 - Status
- Example, printing a character on the printer.
 - Before the CPU sends a character to be printed, it has to first check the **status register** of the associated I/O controller to see whether the printer is online/offline, busy or idle, out of paper, and so on.
 - The **data register** holds the character to be printed
 - The **command register** tells the controller the operation requested by the CPU (e.g., send the character in the data register to the printer).

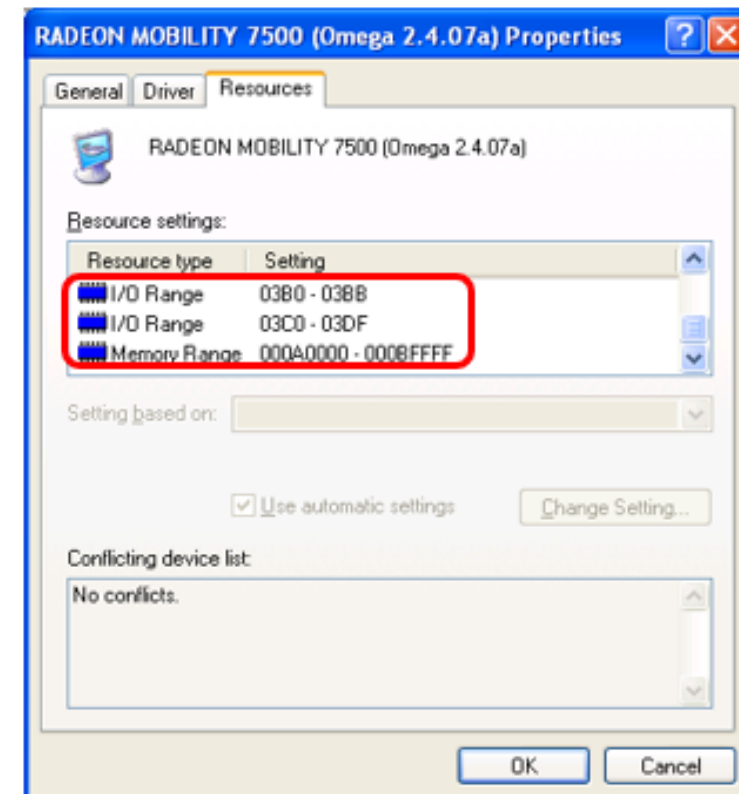
I/O Data Transfer

- I/O requests are made by applications or OS
 - involve moving data between peripheral device and main memory
- To communicate with an I/O device, we need
 - Access to various registers (data, status,...)
 - Memory-mapped I/O
 - Isolated I/O
- A protocol to communicate (to transfer data between devices and main memory)
 - Programmed I/O
 - Interrupt-driven I/O
 - Direct memory access

Accessing I/O Devices

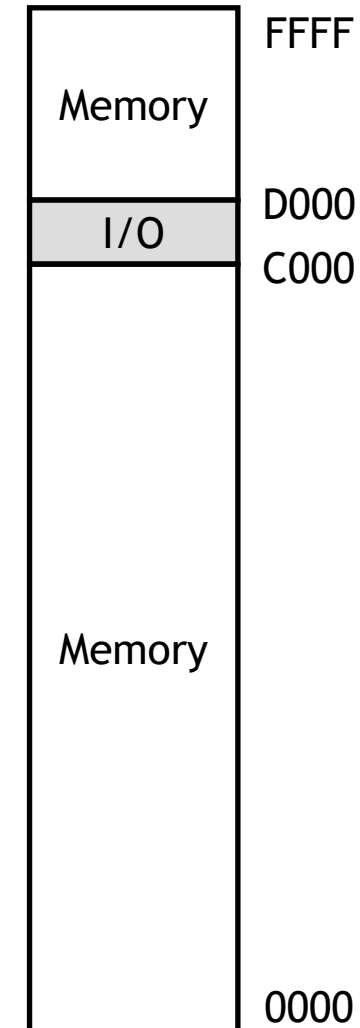
- Most devices can be considered as memories, with an “address” for reading or writing
- Many ISAs* often make this analogy explicit — to transfer data to/from a particular device, the CPU can access special addresses
 - *Example:* Video card can be accessed via addresses 3B0-3BB, 3C0-3DF and A0000-BFFFF
- Two ways to access these addresses:
 - Memory-mapped I/O
 - Isolated I/O

*ISA (Industry Standard Architecture) is a type of computer bus and expansion card standard.



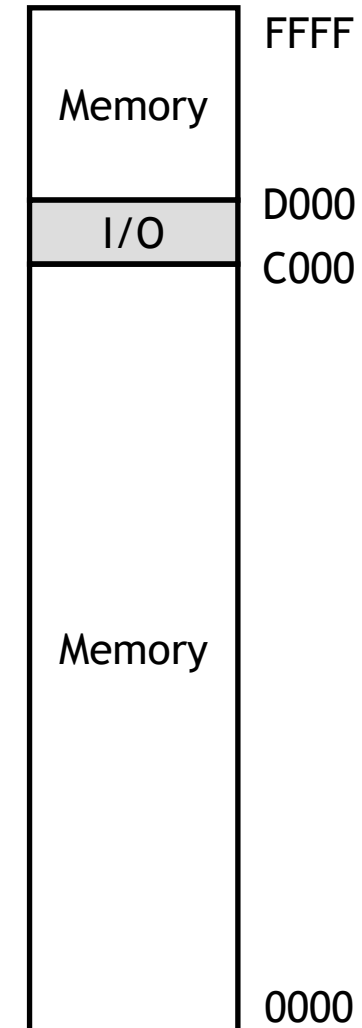
Memory-mapped I/O

- With **memory-mapped I/O**, one address space is divided into two parts.
 - Some addresses refer to physical memory locations.
 - Other addresses actually reference peripherals.
- For example, an old PC with a 16-bit address bus could access a whole 64KB of memory.
 - Addresses **C000-CFFF** in hexadecimal were not part of memory, but were used to access I/O devices.
 - All the other addresses did reference main memory.
- The I/O addresses are shared by many peripherals. For instance, C010 might be attached to the keyboard while C030 goes to the speaker.
- Some devices may need several I/O addresses.



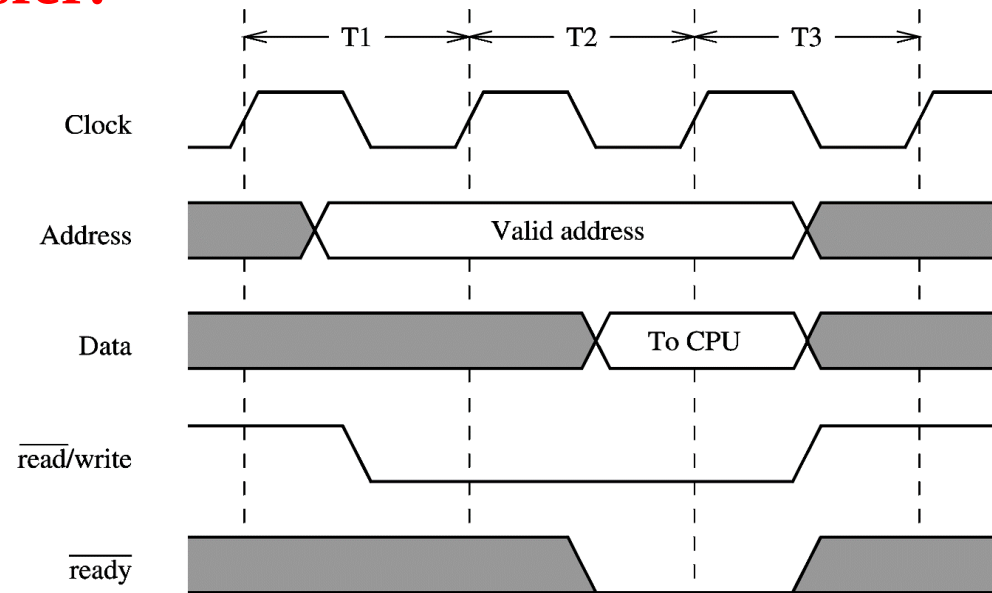
Memory-mapped I/O

- In summary, memory-mapped I/O **maps I/O port address to memory address space**.
 - Writing to an I/O port is similar to writing to a memory location.
 - **Does not require any special consideration** from the processor.
 - However, it **takes part of the memory address space** for I/O mapping.
- Processors such as **MIPS and PowerPC** support only memory-mapped I/O.



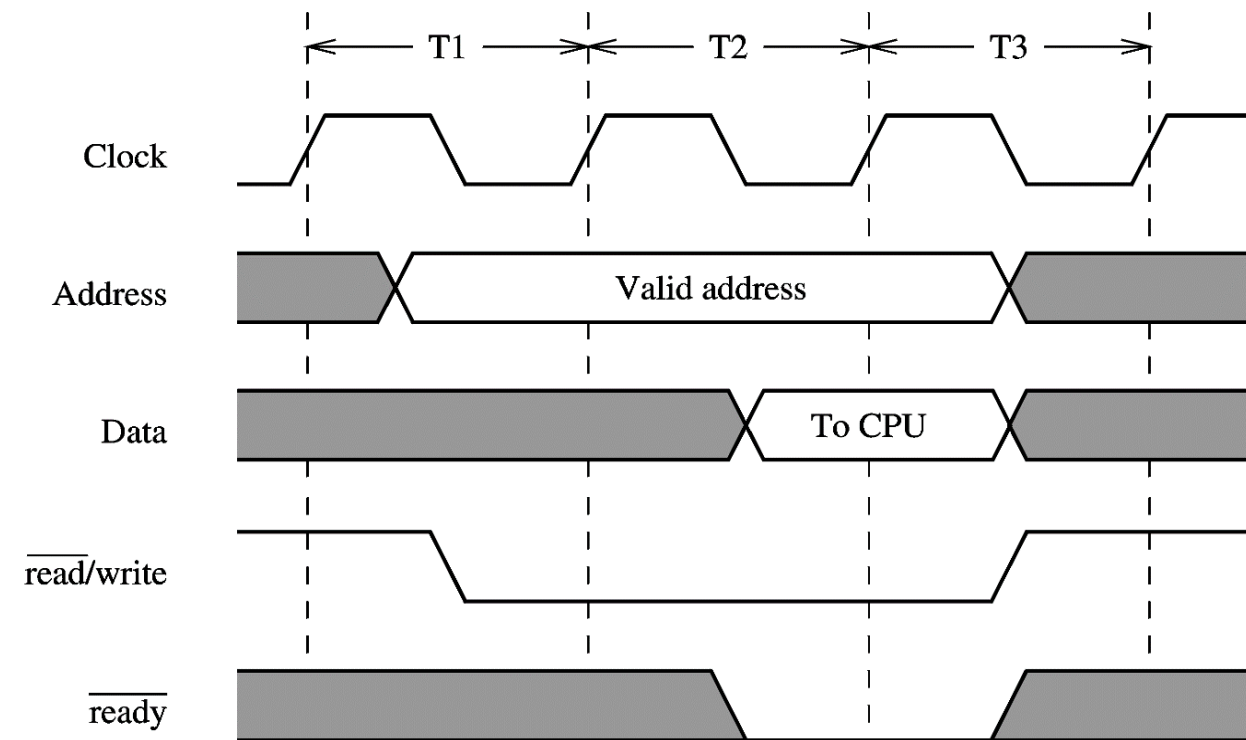
Memory-mapped I/O

- To send a data to a device,
 - The CPU places a **device address** on the **address bus** to activate the path.
 - The CPU provides a **function code** on the **control lines**.
 - The interface proceeds to execute the function and synchronise the **data transfer**.



Synchronous bus (Changes occur relative to the falling or rising edge of the clock)

Memory-mapped I/O



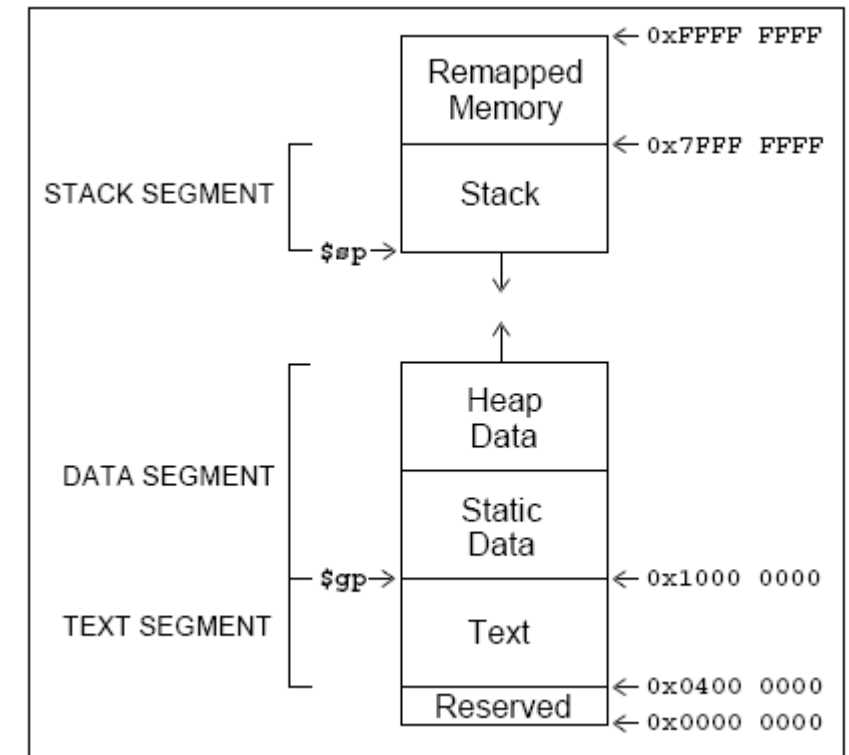
Synchronous bus (Changes occur relative to the falling or rising edge of the clock)

Steps in a Typical Read Cycle:

1. Place the address of the location to be read on the **address bus**
2. Activate the memory read **control signal** on the control bus
3. Wait for the memory to retrieve the data from the addressed memory location and place them on the **data bus**
4. **Read the data** from the data bus
5. Drop the memory read control signal **to terminate the read cycle**

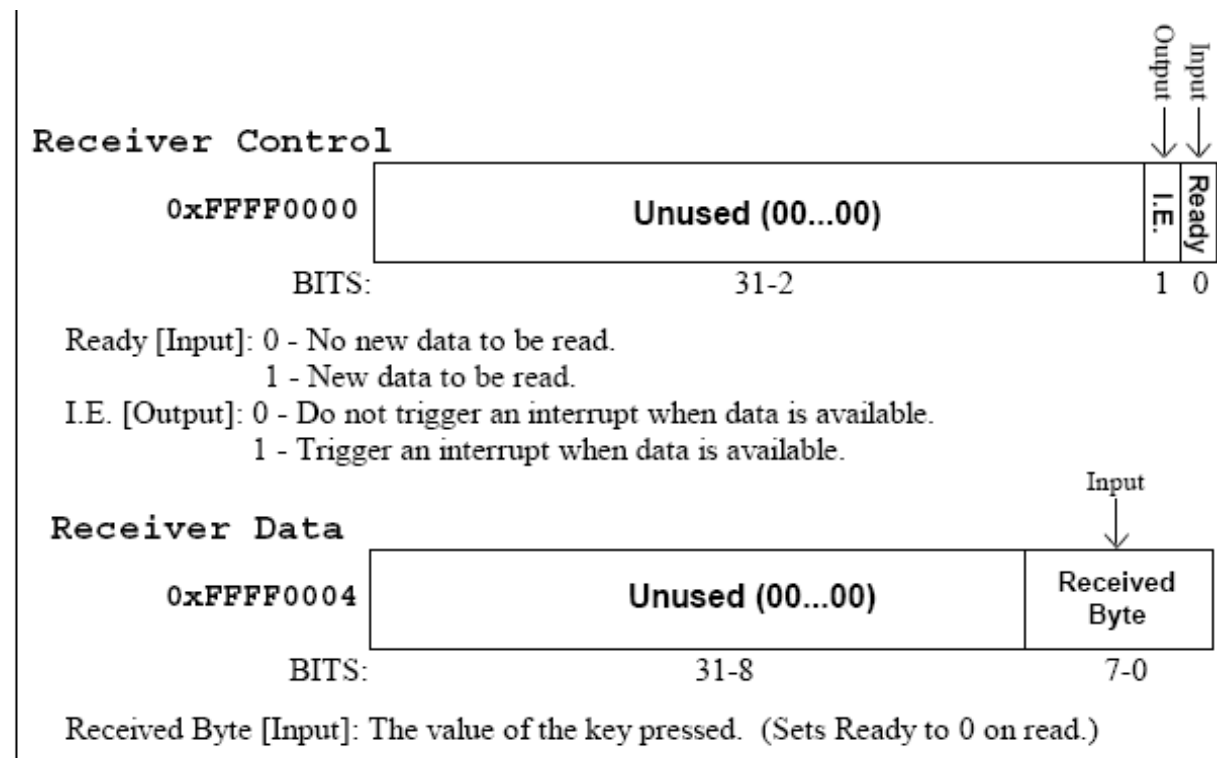
MIPS Memory-mapped I/O

- MIPS processor uses memory-mapped I/O.
- Everything **above 0x7FFF FFFF** is used by the system
 - Special operating system functions
 - I/O registers mapped to memory address
 - Kernel data, etc.



SPIM Input

- SPIM allows you to read from the keyboard (which is similar to reading something from the true I/O register)
- Input is controlled via two memory-mapped device registers:
 - Receiver control
 - Receiver data



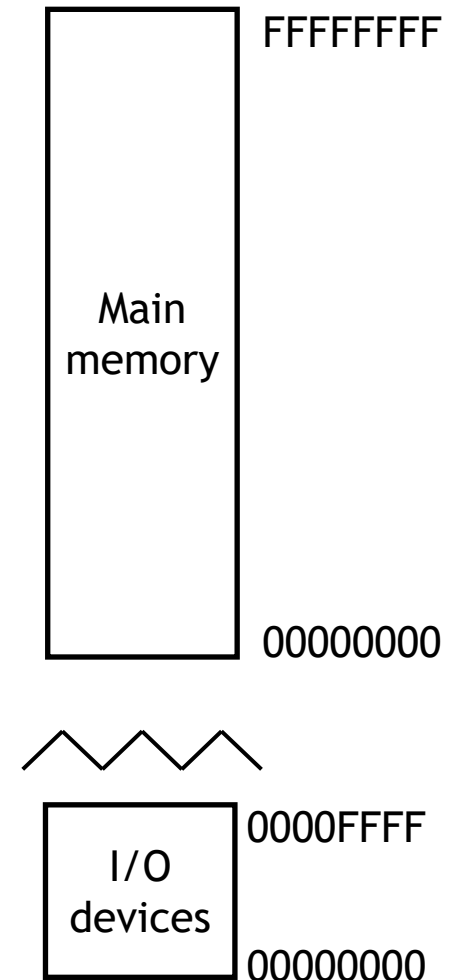
I.E. = Interrupt enable

SPIM Output

- Transmitter writes characters to the display (console).
- Output is controlled via two memory-mapped device registers:
 - **Transmitter control** (0xFFFF 0008)
 - Bit 1: interrupt enable
 - Bit 0: ready
 - **Transmitter data** (0xFFFF 000C)
 - Bit 0-7: data byte
- If you need to show something on the console, do the following:
 1. Check if ready bit is 1. If yes, proceed. Otherwise, wait.
 2. Write to the data. The ready bit will be reset to 0, and will be set to 1 after the byte is transmitted.

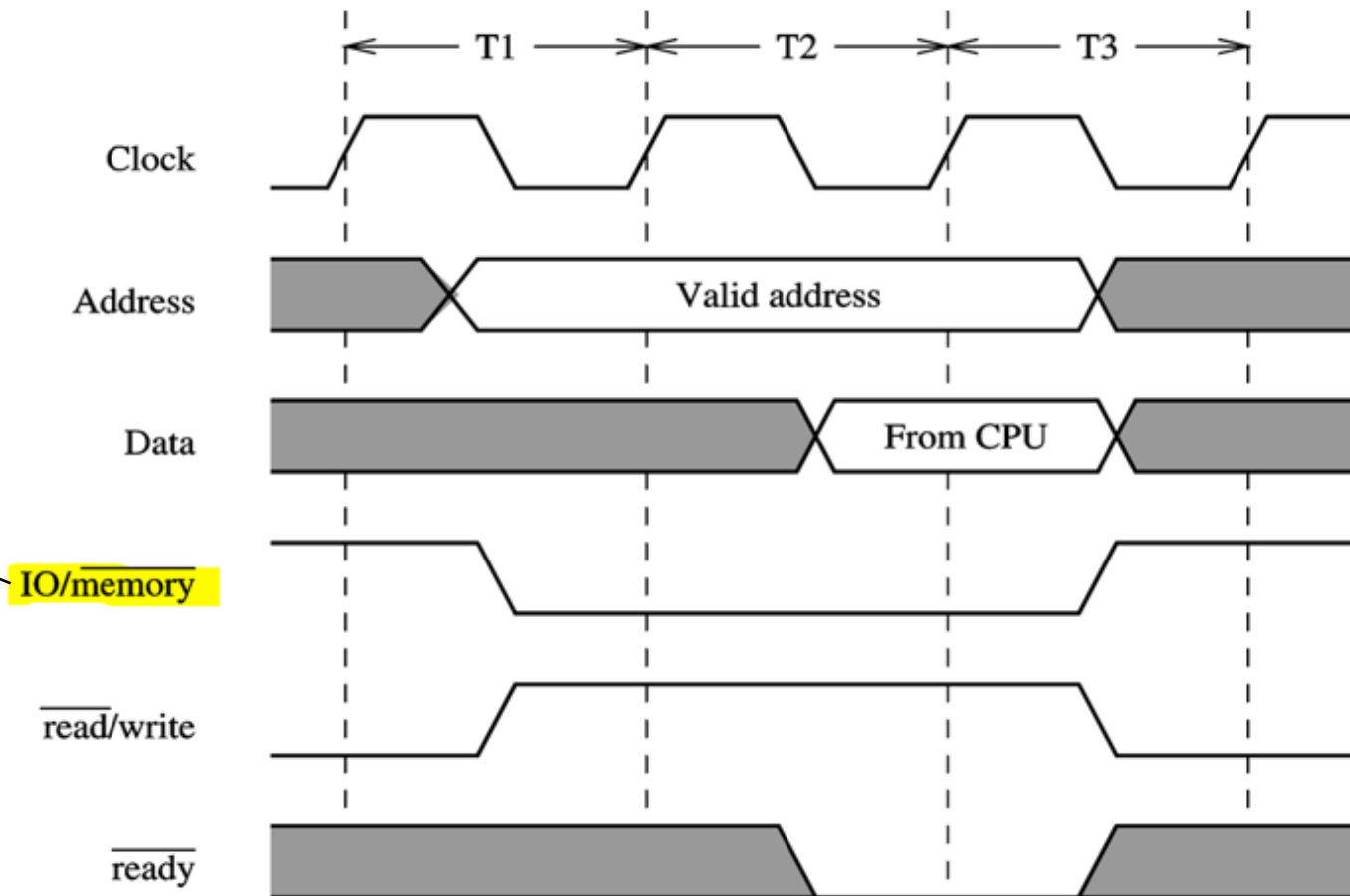
Isolated I/O

- For **isolated I/O**, there are *separate* address spaces for memory and I/O devices
 - special instructions that access the I/O space
- The complete memory address space can be used for memory, but it requires special I/O instructions to access the I/O address space.
- Example (x86):*
 - regular instructions like MOV reference RAM
 - special instructions IN and OUT access a separate I/O address space
- An address could refer to *either* main memory or an I/O device, depending on the instruction used.



Isolated I/O

An address could refer to *either* main memory or an I/O device, depending on the instruction used

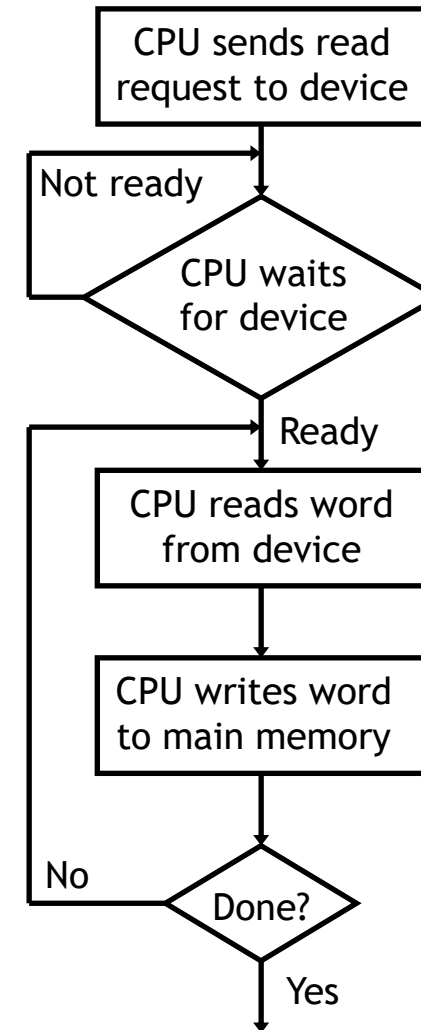


Mode of Data Transfer

- How data is transferred between peripherals and main memory?
- Several modes:
 - Programmed I/O
 - Interrupt-driven I/O
 - Direct memory access

Programmed I/O

- With **programmed I/O**, it's all up to the program or the OS
 - CPU makes a request and then waits (loops) until device is ready (loop 1)
 - Buses are typically 32-64 bits wide, so loop 2 is repeated for large transfers
- A lot of CPU time is needed for this!
 - most devices *are* slow compared to CPUs
 - CPU also “wastes time” doing actual data transfer
- Does not require any hardware support to implement.

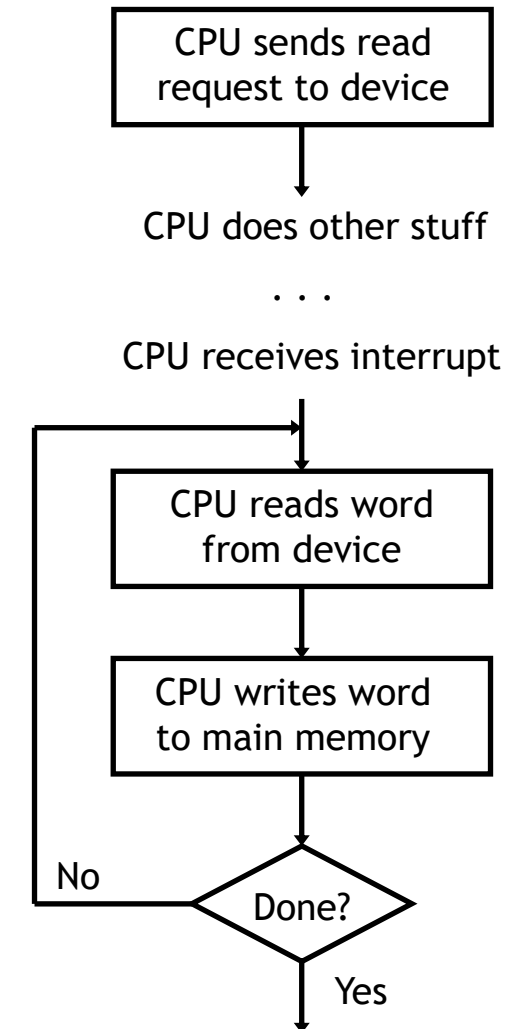


Programmed I/O

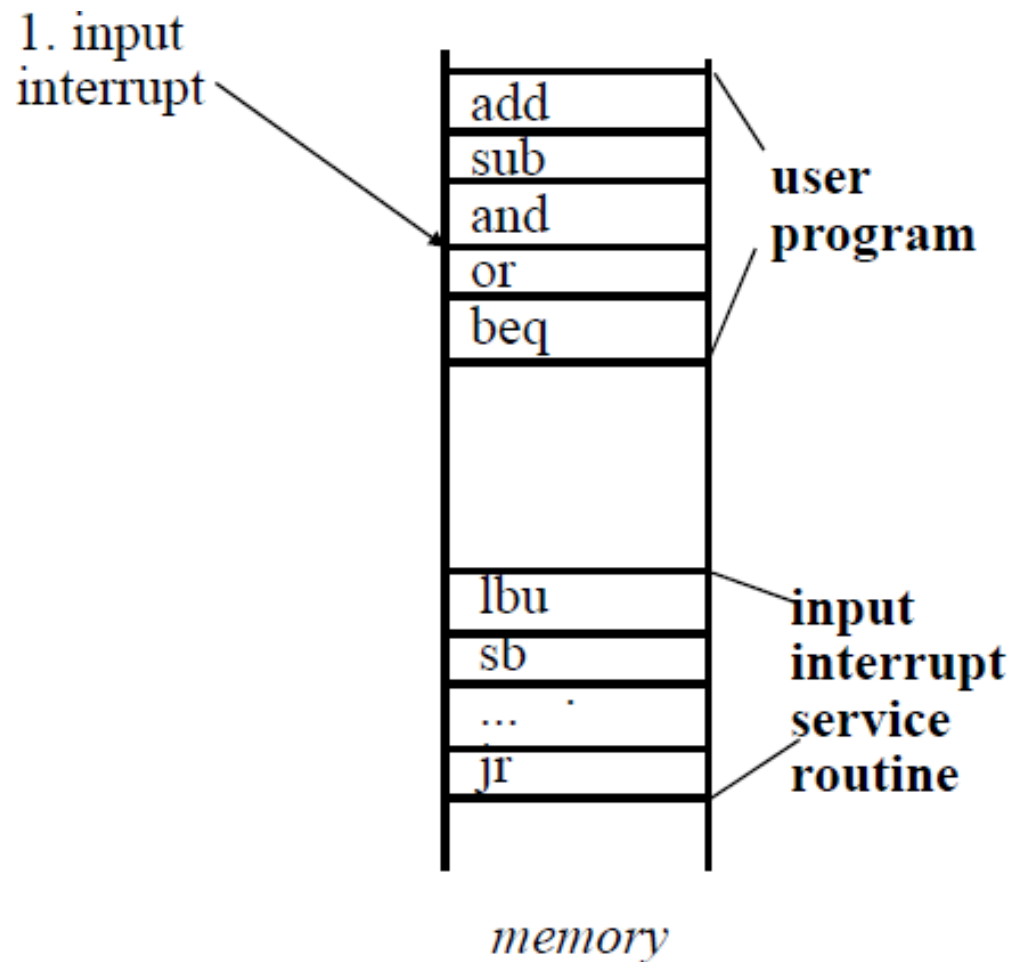
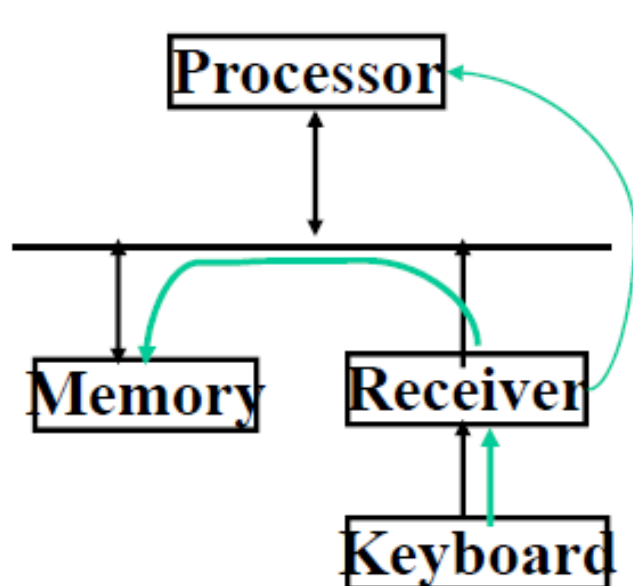
- Continually checking to see if a device is ready is called **polling**
- Wastes CPU time:
 - CPU must **ask repeatedly**
 - CPU must ask often enough to ensure that it doesn't miss anything, which means it **can't do much else while waiting**

Interrupt-driven I/O

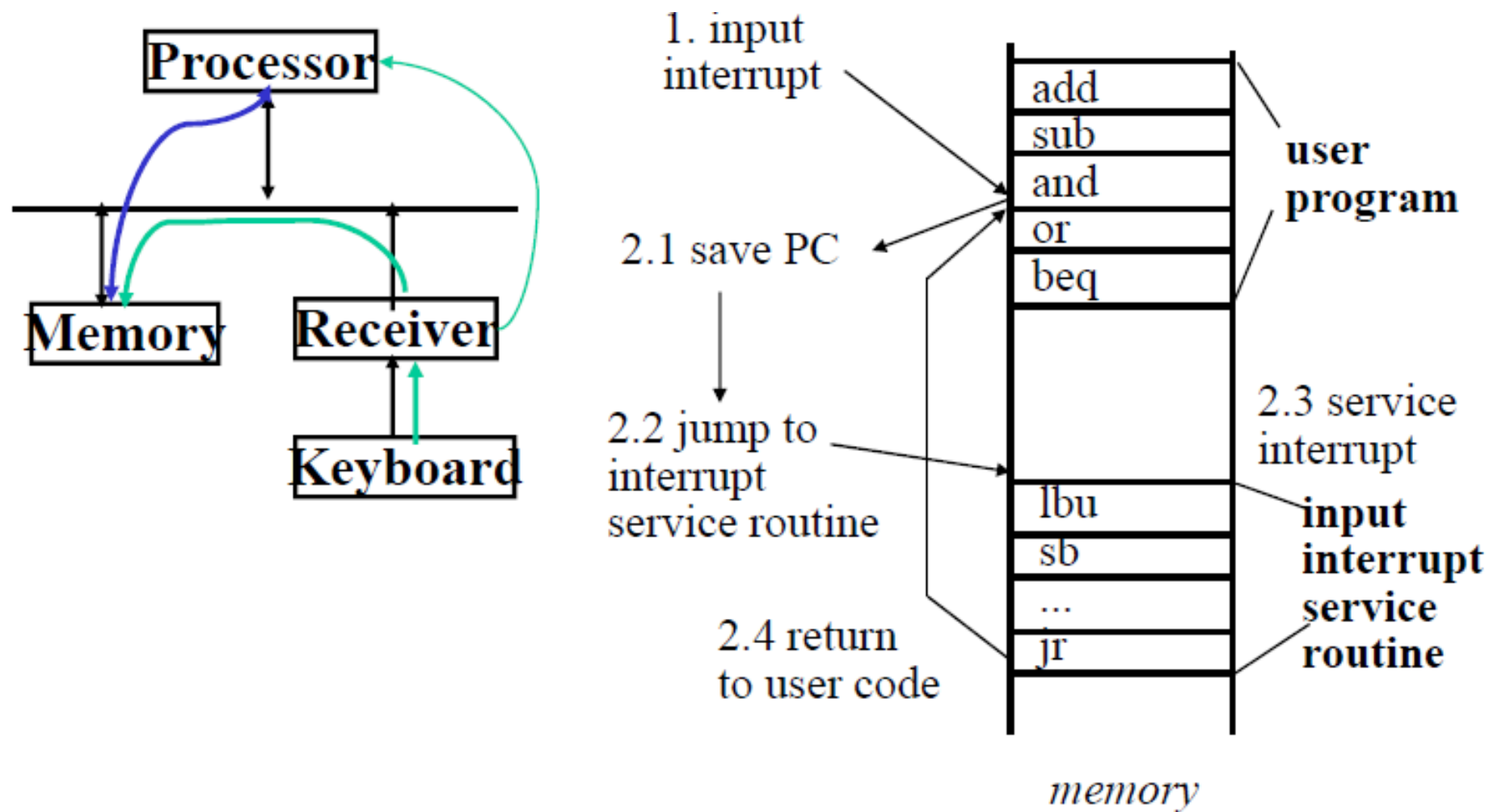
- Instead of waiting, the CPU continues with other calculations
- The device **interrupts** the processor when the data is ready
- CPU still does the data transfer



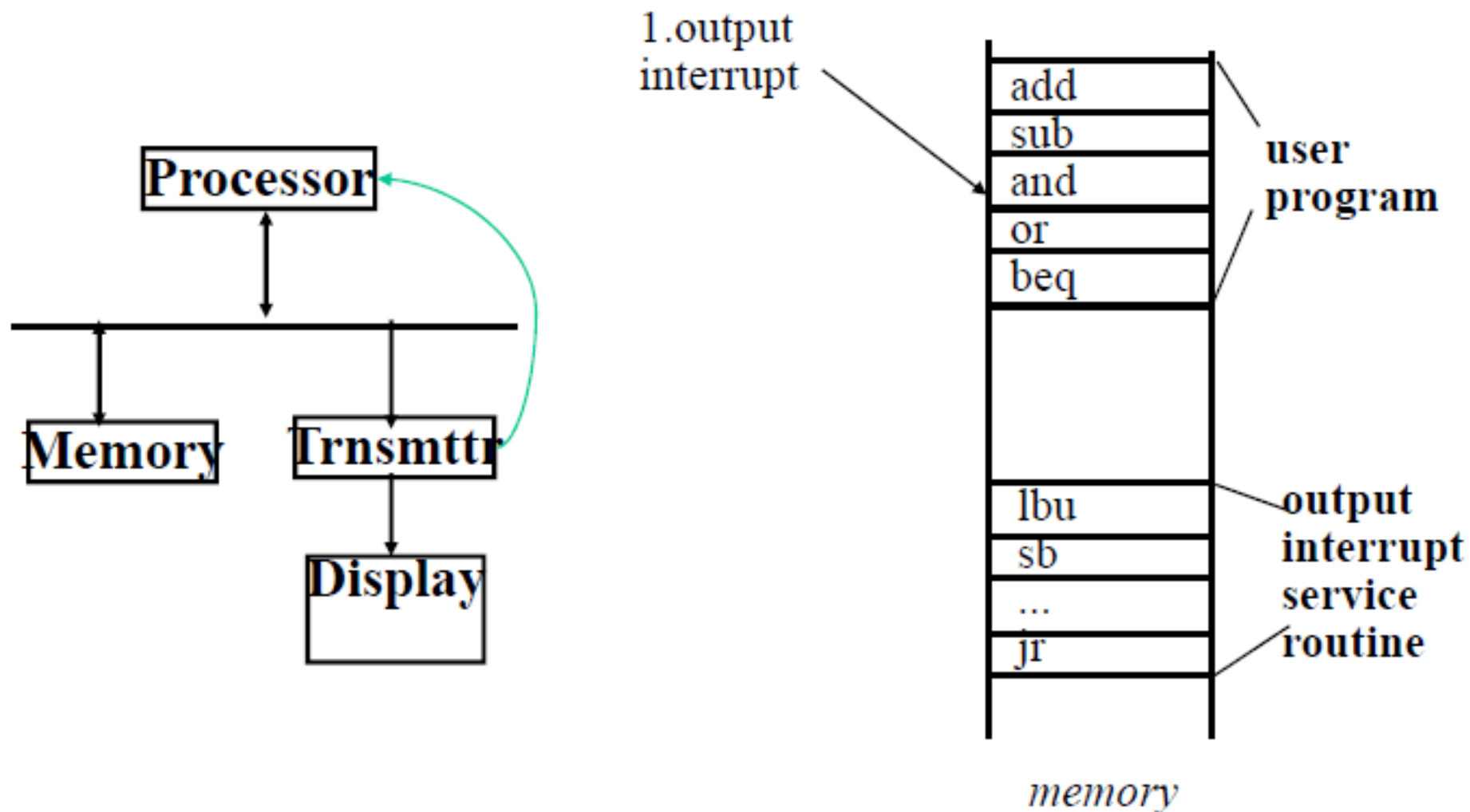
Interrupt-driven Input



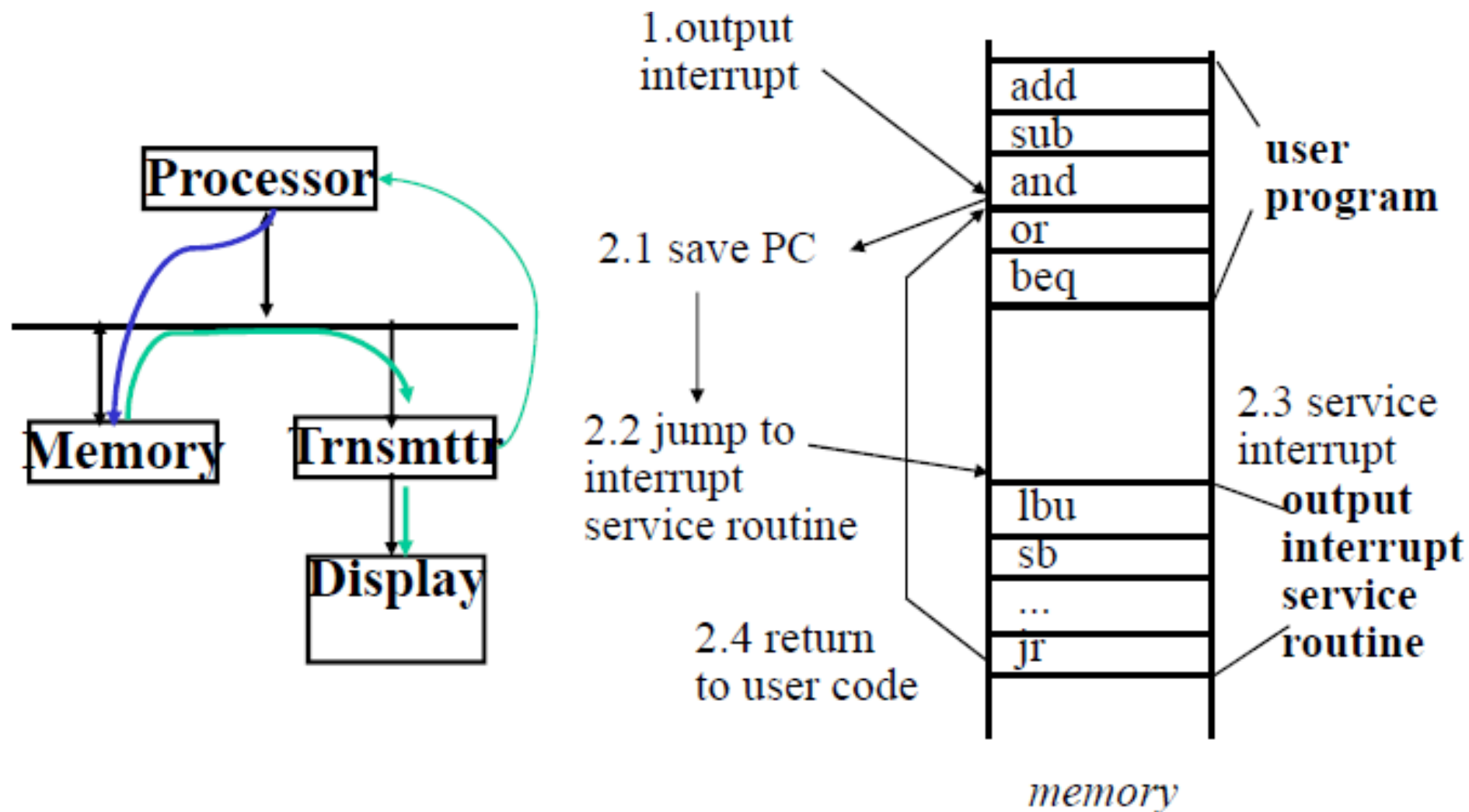
Interrupt-driven Input



Interrupt-driven Output



Interrupt-driven Output





Q & A
Thank you